
Relatório Técnico: API Conecta à Vida

1. Sobre o Projeto

O projeto **Conecta à Vida** é uma aplicação voltada para a gestão de saúde, com foco específico na notificação digital de vacinas e campanhas de doação de sangue. Desenvolvido de forma colaborativa com Gustavo, Gabriel, Renan e Maycon, o sistema back-end atua como o motor principal para processar regras de negócio, gerenciar o banco de dados e fornecer as informações necessárias para as interfaces de usuário (como aplicações mobile ou web). O objetivo central é facilitar o acompanhamento do histórico de saúde dos pacientes e otimizar o fluxo de campanhas e unidades de atendimento.

2. Funcionalidades

A arquitetura do sistema foi dividida em diversos módulos de domínio para abranger todas as necessidades do aplicativo. As principais funcionalidades expostas pela API incluem:

- **Gestão de Pacientes e Unidades de Saúde:** Endpoints para cadastrar, atualizar e consultar dados de pacientes e dos postos/unidades de atendimento.
- **Controle de Vacinas e Registros:** Mapeamento completo do catálogo de vacinas e do histórico de imunização (Registro de Vacinação) de cada usuário.
- **Gerenciamento de Campanhas e Alertas:** Criação de campanhas (como mutirões de vacinação ou doação de sangue) e disparo/gerenciamento de alertas para os usuários.
- **Dashboard e Estatísticas:** Processamento de dados para gerar estatísticas em tempo real (através dos DTOs `DashboardStatsDTO` e `ChartDataDTO`), permitindo a visualização do engajamento e métricas de saúde.
- **Geração de Relatórios:** Exportação de dados consolidados em formato PDF para análise administrativa.
- **Auditoria:** Registro contínuo de atividades do sistema (`LogAtividade`) para garantir segurança e rastreabilidade das ações.

3. O que foi Melhorado

Observando a estrutura atual do projeto, destacam-se várias melhorias e boas práticas de engenharia de software aplicadas:

- **Arquitetura em Camadas:** O código foi rigorosamente separado em Controladores (Controllers), Serviços (Services), Repositórios (Repositories) e Entidades de Domínio (Domains). Isso melhora a manutenção e a divisão de responsabilidades.
- **Implementação de DTOs (Data Transfer Objects):** O uso de DTOs nas respostas do Dashboard garante que a API envie apenas os dados estritamente necessários para o front-end, economizando banda e protegendo dados sensíveis das entidades do banco.

- **Geração Nativa de Documentos:** A integração com a biblioteca iText7 representa uma melhoria significativa, permitindo que a aplicação não apenas gerencie dados, mas também gere relatórios profissionais e formatados em PDF diretamente pelo back-end.
- **Mapeamento Objeto-Relacional Automático:** O uso do Hibernate configurado com `ddl-auto=update` acelera o desenvolvimento, pois o banco de dados evolui automaticamente junto com as mudanças feitas nas classes de domínio Java.

4. Tecnologias Utilizadas

A stack tecnológica escolhida é moderna e robusta, baseada no ecossistema Java:

- **Linguagem:** Java 17.
- **Framework Principal:** Spring Boot (versão 4.0.5).
- **Mapeamento e Persistência:** Spring Data JPA e Hibernate.
- **Banco de Dados:** PostgreSQL (driver `org.postgresql:postgresql`).
- **Web e APIs:** Spring Boot Starter WebMVC para criação dos endpoints REST.
- **Produtividade:** Lombok (para redução de código repetitivo, como Getters/Setters).
- **Relatórios:** iText7 Core (versão 7.2.5) para manipulação de PDFs.
- **Automação de Build:** Gradle.

5. Como a aplicação "puxa" os dados (Fluxo de Funcionamento)

A aplicação utiliza o padrão MVC (Model-View-Controller) adaptado para APIs REST. O fluxo de dados ocorre da seguinte forma:

1. **Conexão:** Ao iniciar, o Spring Boot lê o arquivo `application.properties` e estabelece uma conexão JDBC com o PostgreSQL hospedado em `localhost:5432`, acessando o banco `conecta_a_vida_db` com as credenciais configuradas.
2. **Requisição (Controller):** Quando um aplicativo front-end faz uma requisição HTTP (por exemplo, pedir os dados do Dashboard), o Controlador responsável (ex: `DashboardController`) intercepta a chamada.
3. **Regra de Negócio (Service):** O Controlador repassa a ordem para a camada de Serviço. Aqui, a lógica de negócio é validada e calculada.
4. **Busca no Banco (Repository):** O Serviço aciona o Repositório (que herda do Spring Data JPA) para fazer a consulta. O Hibernate traduz essa requisição para código SQL automaticamente e puxa os dados das tabelas do PostgreSQL.
5. **Retorno:** Os dados viajam de volta, são empacotados em um arquivo JSON pelo Spring Web e entregues ao aplicativo.

6. Como Instalar e Executar

Para rodar a aplicação localmente para desenvolvimento:

1. Instale o **Java JDK 17** e configure as variáveis de ambiente.
2. Instale o **PostgreSQL**.
3. Abra sua ferramenta de banco de dados (ex: `pgAdmin`) e crie um banco de dados vazio

chamado exatamente conecta_a_vida_db.

4. Certifique-se de que a senha do usuário postgres no seu computador seja 20040124 (ou altere o arquivo application.properties para corresponder à sua senha local).
5. Pelo terminal, dentro da pasta raiz do projeto, execute o Gradle Wrapper para baixar dependências e subir o servidor:
 - No Windows: gradlew.bat bootRun
 - No Linux/Mac: ./gradlew bootRun

7. Possíveis Erros e Soluções

- **Connection to localhost:5432 refused ou FATAL: password authentication failed: ***
Causa: O banco de dados PostgreSQL não está rodando, a porta está errada, ou a senha 20040124 configurada no arquivo application.properties não bate com a do seu banco local.
 - *Solução:* Verifique se o serviço do Postgres está ativo e corrija as credenciais no código se necessário.
- **Web server failed to start. Port 8080 was already in use.: *** *Causa:* Outro programa (ou outra instância da mesma API) já está utilizando a porta padrão do Spring Boot.
 - *Solução:* Derrube o processo que está usando a porta ou adicione a linha server.port=8081 no application.properties.
- **Erros de "Símbolo não encontrado" (Get/Set) na IDE: *** *Causa:* O código compila no terminal, mas a IDE acusa erro nas classes de Domínio porque o **Lombok** não está ativado.
 - *Solução:* Instale o plugin do Lombok na sua IDE (Eclipse, IntelliJ ou VS Code) e habilite o "Annotation Processing".
- **UnsupportedClassVersionError: *** *Causa:* Tentativa de rodar o projeto com uma versão do Java anterior à 17.
 - *Solução:* Atualize a variável JAVA_HOME para apontar para o JDK 17.